We have to whitelist our controller parameters to prevent wrongful mass assignment. In this case, we want to both allow and require the `title` and `text` parameters for valid use of `create`. The syntax for this introduces `require` and `permit`. The change will involve one line in the `create` action:

```
@article = Article.new(params.require(:article).permit(:title, :text))
```

This is often factored out into its own method so it can be reused by multiple actions in the same controller, for example `create` and `update`. Above and beyond mass assignment issues, the method is often made `private` to make sure it can't be called outside its intended context. Here is the result:

```
def create
  @article = Article.new(article_params)

  @article.save
  redirect_to @article
end

private
  def article_params
    params.require(:article).permit(:title, :text)
  end
```

For more information, refer to the reference above and this blog article about Strong Parameters.

5.7 Showing Articles

If you submit the form again now, Rails will complain about not finding the `show` action. That's not very useful though, so let's add the `show` action before proceeding.

As we have seen in the output of `rake routes`, the route for `show` action is as follows:

```
article GET    /articles/:id(.:format)      articles#show
```

The special syntax `:id` tells rails that this route expects an `:id` parameter, which in our case will be the id of the article.

As we did before, we need to add the `show` action in `app/controllers/articles_controller.rb`and its respective view.

A frequent practice is to place the standard CRUD actions in each controller in the following order: `index`, `show`, `new`, `edit`, `create`, `update` and `destroy`. You may use any order you choose, but keep in mind that these are public methods; as mentioned earlier in this guide, they must be placed before any private or protected method in the controller in order to work.

Given that, let's add the `show` action, as follows:

```
class ArticlesController < ApplicationController
  def show
    @article = Article.find(params[:id])
  end

  def new
  end

  # snipped for brevity
```

A couple of things to note. We use `Article.find` to find the article we're interested in, passing in `params[:id]` to get the `:id` parameter from the request. We also use an instance variable (prefixed with `@`) to hold a reference to the article object. We do this because Rails will pass all instance variables to the view.

Now, create a new file `app/views/articles/show.html.erb` with the following content:

```
<p>
  <strong>Title:</strong>
  <%= @article.title %>
</p>

<p>
```